

Programming Thinking & Python Basics

Session 07 practical workbook · Python 3 · VS Code · Sales quote utility

This is the hands-on companion to the Session 07 slides. The slides explain the ideas; this workbook puts every idea into a small piece of code you type and run in VS Code. One business problem runs through the whole session: a small business needs a **sales quote**. By the end, learners will have built a working quote calculator as a single `.py` file.

Every example was run on Python 3 and the output printed here is the real output, including the error messages. Exact wording of errors can differ slightly between Python versions; the error *type* on the last line will be the same.

Part	Topic	What you do
	Before you start	Check that VS Code can find and run Python
1	Think before coding	Turn the quote request into inputs, steps and outputs; run your first script
2	Where Python runs	Run a file, use the terminal, and see how a REPL keeps state
3	Values, statements and variables	Assign, print, reassign; name things well
4	Core types and operators	Use int, float, str and bool; calculate and compare
5	Input and conversion	Read from the keyboard and turn text into numbers
6	Strings and indexing	Clean names, count characters, pick letters, format output
7	Reading errors and debugging	Read a traceback and fix five common errors
8	Build the quote calculator	Combine everything into one working program
9	Variation and close	Add a service fee; explain your program

Session mental model. Keep coming back to this sequence. Every part of the session is one piece of it.



Lesson plan (3 hours)

Use the slides only to frame each block, then move into VS Code quickly. A rough split: about 10 minutes of slides per block, the rest live coding and learner practice. The relevant slide numbers are shown so you can jump straight to them.

Time	Block	You do (trainer)	Learners do	Slides
0-15	Part 1 · Think before coding	Read the quote request aloud. Build the inputs/steps/outputs table with the class. Live-code <code>p1_quote_preview.py</code> .	Predict 250×4 before running. Run their first file.	1-7
15-30	Part 2 · Where Python runs	Show ▶ Run, the terminal, and Shift+Enter. Do the state demo once.	Run one file three ways. See the REPL remember a value.	8-15
30-55	Part 3 · Values and variables	Type each example; ask for a prediction before every run.	Predict outputs; Try it yourself 3.	16-25
55-80	Part 4 · Types and operators	Make <code>"4"</code> vs <code>4</code> visible. Contrast <code>=</code> and <code>==</code> out loud.	Try it yourself 4 (coffee order maths).	26-37
80-90	Break			38
90-115	Parts 5-6 · Input, conversion, strings	Real <code>input()</code> in the terminal. Index table on the board.	Try it yourself 5 and 6.	39-49
115-140	Part 7 · Debugging	Model the loop on one error. Let them do the other stations.	Five stations; Try it yourself 7.	50-60
140-172	Parts 8-9 · Build the calculator	Stop lecturing. Circulate and coach. Check intermediate values.	Complete the starter file; defect hunt; service-fee variation.	61-69
172-180	Close	Ask learners to point at each concept in their own file.	Explain your program checklist.	70-72

Before class

- ☐ VS Code installed with the **Python** extension (publisher: Microsoft).
- ☐ Python 3.10 or newer installed. Open a VS Code terminal and run `python --version` (Windows may need `py --version`, macOS `python3 --version`).
- ☐ A folder called `s07_python_basics` opened in VS Code with **File > Open Folder**.
- ☐ The starter files from the class bundle copied into that folder (`quote_calculator_starter.py`, `debug_station_1.py` ... `debug_station_5.py`, `try_it_debug.py`).
- ☐ Ask learners to send a screenshot of `python --version` before the session, so installation problems are fixed before class time.

Teaching moves that make this work

Move	How it sounds in class
Predict before running	"Write down what you think this prints. Now run it. Were you right?"
Read assignment correctly	Say <code>quantity = 4</code> as "assign 4 to quantity", never "quantity equals 4".
Output is evidence	"The terminal shows a copy of the value. It is not the code and not the variable."
Errors are information	"What does the last line of the error say? Which line number?" before anyone touches the code.
One change at a time	"Change one thing, run again. If it's still broken, you learned something."
Park, don't pre-teach	"Great question, that's <code>if</code> statements in Session 08." Then move on.

Trainer note: Learners type the code themselves. Copying from the PDF hides typos, and typos are where most of today's learning about errors comes from.

Before you start: quick checks

Everything today runs from `.py` files in one folder. Do these three checks first. They take two minutes and catch almost every setup problem.

Check 1: Python is installed. In VS Code open the terminal with **Terminal > New Terminal** (or `Ctrl+``) and type:

TERMINAL

```
> python --version
Python 3.12.4
```

Any version from 3.10 up is fine. If you see *"python is not recognized"* on Windows, try `py --version`; on macOS try `python3 --version`. Use whichever word works for the rest of the session.

Check 2: VS Code is using that Python. Press `Ctrl+Shift+P`, type **Python: Select Interpreter** and pick the Python 3 version from the list. The chosen version then appears in the bottom-right corner of the VS Code window.

Check 3: a file runs. Create a new file called `check_setup.py`, type the two lines below, save with `Ctrl+S`, then click the ► **Run Python File** button in the top-right corner.

check_setup.py

```
print("Python is working")
print(2 + 3)
```

OUTPUT

```
Python is working
5
```

Watch out: Always save before you run. A white dot on the file's tab means unsaved changes. Turning on **File > Auto Save** removes this problem for the whole session.

Part 1 Think before coding

A small business asks for this:

"Calculate a quote from unit price, quantity, a discount percentage and a tax percentage. Show the total, and flag whether it is a large order."

A person can fill in the gaps in that sentence. A computer cannot. Does the discount apply before or after tax? What counts as "large"? Before writing any Python we decide exactly what goes in, what happens, and what comes out.

Decompose the problem

Step of the model	For the sales quote
Problem	Prepare a sales quote for one product
Inputs	customer name, product name, unit price, quantity, discount %, tax %, large-order threshold
Steps / Rules	subtotal → discount → discounted subtotal → tax → final total → compare with threshold
Output	a readable summary, the final total, True/False for large order, a short order code
Test / Correct	work out a simple case by hand and check every intermediate value

A plain-language algorithm first

1. Receive the customer, product, price, quantity, discount and tax details.
2. Clean up the customer and product names.
3. Convert numbers typed as text into real numbers.
4. Calculate subtotal, discount, discounted subtotal, tax and final total, in that order.
5. Compare the final total with the large-order threshold.
6. Build an order code from the first letters and the quantity.
7. Display a readable summary.
8. Test with values whose answer you can check by hand.

Trainer note: Build the table on the board with the class before showing the slide version. Ask: "What does the computer need to know?" and "In what order?" Briefly connect to earlier modules: in a spreadsheet this is a cell formula, in SQL it is a query; in Python it is a list of instructions run in order.

Your first script

PREDICT FIRST

If the unit price is 250 and the quantity is 4, what subtotal should the program print? Write it down before running.

Create `p1_quote_preview.py` and type:

p1_quote_preview.py

```
unit_price = 250.0
quantity = 4
subtotal = unit_price * quantity
print(subtotal)
```

OUTPUT

1000.0

Four lines, run from top to bottom: two values are given names, a calculation produces a new value, and `print()` shows it. Notice the result is `1000.0`, not `1000`. We will see why in Part 4.

TRY IT YOURSELF

A café sells coffee at **32.50** per cup. A customer orders **3** cups and adds a **10%** tip.

(a) List the inputs, the steps (in order) and the outputs. (b) Work out the total by hand. You will code this in Part 4.

Part 2 Where Python runs

Three different things are often mixed up. Keeping them apart avoids a lot of confusion later.

1 • Language

Python itself: the rules for writing instructions and the interpreter that runs them.

2 • Runtime

A running Python process plus its installed packages. In VS Code this is the *interpreter* shown in the bottom-right corner (it may come from python.org or from Anaconda).

3 • Interface

Where you write and run code: a `.py` file in the editor, the terminal, or a notebook (`.ipynb`), which VS Code can also open.

Anaconda, Jupyter and Colab are not different languages. Anaconda is a way of installing Python with data-science tools; Jupyter and Colab are notebook interfaces. The Python code is identical in all of them. Today we use VS Code and plain `.py` files.

Three ways to run code in VS Code

Method	How	What happens
Run the file	▶ button, top right	The whole file runs from line 1 to the end. Every run starts fresh.
Terminal command	<code>python</code> <code>p1_quote_preview.py</code>	Exactly the same as the ▶ button. This is how scripts run on servers.
Run selection	Select lines, press Shift+Enter	Sends only those lines to a Python REPL in the terminal. The REPL remembers variables between sends, like a notebook.

A script only shows what you print

PREDICT FIRST

This file has two lines that both work out $2 + 3$. How many numbers will appear in the terminal?

p2_print_only.py

```
2 + 3
print(2 + 3)
```

OUTPUT

5

Only one. The first line calculates `5` and throws it away. In a script, nothing appears unless you `print()` it. In a notebook or the REPL, the value of a bare expression is shown automatically, which is why notebook code sometimes works without `print()` and the same code in a `.py` file appears to do nothing.

The REPL: quick experiments

Type `python` in the terminal and press Enter. The `>>>` prompt means Python is waiting for one line at a time. Type `exit()` to leave.

TERMINAL

```
> python
>>> 250.0 * 4
1000.0
>>> quantity = 4
>>> quantity
4
>>> exit()
```

State: what the running Python remembers

Create `p2_state_demo.py`:

p2_state_demo.py

```
state_demo_price = 100.0
print("Current price:", state_demo_price)

state_demo_price = 125.0
print("Current price:", state_demo_price)
```

OUTPUT

```
Current price: 100.0
Current price: 125.0
```

Run with **▶**: always `100.0` then `125.0`, because a file always runs top to bottom. Now use Shift+Enter instead. Select lines 1-2 and send them, then lines 4-5, then **only line 2** again:

TERMINAL (REPL)

```
>>> state_demo_price = 100.0
>>> print("Current price:", state_demo_price)
Current price: 100.0
>>> state_demo_price = 125.0
>>> print("Current price:", state_demo_price)
Current price: 125.0
>>> print("Current price:", state_demo_price)
Current price: 125.0
```

Line 2 now prints `125.0` even though it sits *above* the line that set 125. The REPL follows the order you **ran** things in, not the order they appear on the page. Notebooks behave exactly the same way.

Tip: To clear the remembered state, close the REPL terminal with the bin icon, or type `exit()`. Then run the whole file with ▶. A clean top-to-bottom run is the test that your program really works.

Part 3 Values, statements and variables

Read every line of Python as **value flow**: where does a value come from, what name does it get, and where is it used next?

Reading an assignment

Take the line `unit_price = 250.0` apart:

Piece	Name	Meaning
<code>unit_price</code>	variable name	A label you chose so you can use the value later
<code>250.0</code>	literal	A value written directly in the code
<code>=</code>	assignment operator	Store the value on the right under the name on the left
the whole line	statement	One instruction Python carries out

Python always works out the **right-hand side first**, then attaches the result to the name on the left. Read the line as "assign 250.0 to unit_price". Do not read `=` as "equals": Python uses `==` for that, in Part 4.

print() shows a copy of a value

p3_print.py

```
unit_price = 250.0
print(unit_price)
print("Unit price:", unit_price)
```

OUTPUT

```
250.0
Unit price: 250.0
```

`print()` can take several values separated by commas; it puts a space between them. What appears in the terminal is evidence about the program's state. It is not the code, and changing what you see in the terminal changes nothing in the program.

A name versus text

PREDICT FIRST

Two of these lines look almost the same. What will each one print?

p3_name_vs_text.py

```
customer_name = "Maya Patel"
print(customer_name)
print("customer_name")
```

OUTPUT

```
Maya Patel
customer_name
```

Without quotes, `customer_name` is a name: Python looks up the value stored under it. With quotes, `"customer_name"` is just text, printed exactly as written.

Reassignment

PREDICT FIRST

What are the two printed values?

p3_reassign.py

```
practice_quantity = 4
print(practice_quantity)

practice_quantity = 5
print(practice_quantity)
```

OUTPUT

```
4
5
```

The second assignment does not create a second `practice_quantity`. It replaces the value from that line onwards. Because the right side is worked out first, a name can even be used to update itself:

p3_update.py

```
quantity = 4
quantity = quantity + 1
print(quantity)
```

OUTPUT

```
5
```

Python reads the current `quantity` (4), adds 1, then stores 5 back under `quantity`. As maths this line would be impossible; as an instruction it makes perfect sense.

Expressions combine values

An **expression** is any piece of code that produces a value: `250.0`, `unit_price * quantity`, `discount_pct / 100`. Expressions can be built from other expressions:

p3_expressions.py

```

unit_price = 250.0
quantity = 4
discount_pct = 10.0

subtotal = unit_price * quantity
discount_amount = subtotal * (discount_pct / 100)
print(subtotal)
print(discount_amount)

```

OUTPUT

```

1000.0
100.0

```

After this line	Value now stored
unit_price = 250.0	unit_price → 250.0
quantity = 4	quantity → 4
subtotal = unit_price * quantity	subtotal → 1000.0
discount_amount = subtotal * (discount_pct / 100)	first 10.0 / 100 → 0.1, then 1000.0 * 0.1 → discount_amount → 100.0

Naming rules

Rule	Fine	Not allowed
Letters, digits and underscores only	unit_price	unit price, unit-price
Cannot start with a digit	price_2	2nd_price
Case-sensitive	unit_price and Unit_Price are two different names	
Not a Python keyword	class_name	class, True, if

By convention Python uses **snake_case**: lowercase words joined by underscores. Compare `x = p * q` with `subtotal = unit_price * quantity`. The second explains itself.

p3_bad_name.py

```
2nd_price = 300.0
```

ERROR

```

File "C:\Users\student\s07_python_basics\p3_bad_name.py", line 1
  2nd_price = 300.0
    ^
SyntaxError: invalid decimal literal

```

p3_case.py

```

unit_price = 250.0
print(Unit_Price)

```

ERROR

```

Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\p3_case.py", line 2, in <module>
    print(Unit_Price)
          ^^^^^^^^^
NameError: name 'Unit_Price' is not defined. Did you mean: 'unit_price'?

```

Newer Python versions even suggest the name you probably meant.

Comments and indentation

p3_comments.py

```
# Calculate the subtotal before any discount
subtotal = 250.0 * 4
print(subtotal) # a comment can also follow code on the same line
```

OUTPUT

```
1000.0
```

Python ignores everything after `#`. Good comments explain *why*, not what an already clear line does. For today, every statement starts at the left margin. Leading spaces mean something in Python (they group code into blocks, which you will use in Session 08), so an accidental indent is an error. You will see it in Part 7.

TRY IT YOURSELF

(a) Predict the output, then run it:

```
cups = 2 / cups = cups + 3 / print(cups) / print("cups")
```

(b) Which of these are valid variable names? `coffee price`, `coffee_price`, `3cups`, `cups3`, `class`, `Cups`

Part 4 Core types and operators

Every value has a **type**, and the type decides what you can do with it. Today we need exactly four.

Type	Holds	Examples in the quote
<code>int</code>	whole numbers	<code>quantity = 4</code>
<code>float</code>	decimal numbers	<code>unit_price = 250.0, tax_pct = 5.0</code>
<code>str</code>	text, in quotes	<code>customer_name = "Maya Patel"</code>
<code>bool</code>	<code>True</code> or <code>False</code>	<code>is_large_order = 945.0 >= 950.0</code>

`type()` tells you the type of any value. Use it while learning, when a type is not obvious.

p4_types.py

```

quantity = 4
unit_price = 250.0
customer_name = "Maya Patel"
is_large_order = 945.0 >= 950.0

print(type(quantity))
print(type(unit_price))
print(type(customer_name))
print(type(is_large_order))

```

OUTPUT

```

<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>

```

Ignore the word `class` in the output for now. Read `<class 'int'>` simply as "this value is an int".

4 is not "4"

PREDICT FIRST

Both lines multiply "four" by 250.0. Will both work?

p4_four.py

```

print(4 * 250.0)
print("4" * 250.0)

```

OUTPUT

```
1000.0
```

ERROR

```

Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\p4_four.py", line 2, in <module>
    print("4" * 250.0)
          ~~~~^~~~~~
TypeError: can't multiply sequence by non-int of type 'float'

```

On screen `4` and `"4"` look the same, but Python sees a number and a piece of text. Text cannot be multiplied by a decimal, so Python stops with a `TypeError`. Some text operations do work, but they are not maths:

p4_text_ops.py

```

print("4" * 3)
print("4" + "4")

```

OUTPUT

```
444
44
```

Watch out: `"4" * 3` repeats the text three times and `"4" + "4"` joins two pieces of text. Neither one gives an error, so a missing conversion can produce a wrong answer without any warning. This matters in Part 5, where every keyboard input arrives as text.

Arithmetic

Operator	Meaning	Example	Result
+	add	900.0 + 45.0	945.0
-	subtract	1000.0 - 100.0	900.0
*	multiply	250.0 * 4	1000.0
/	divide	10 / 100	0.1

p4_arith.py

```
print(250 * 4)
print(250.0 * 4)
print(8 / 2)
```

OUTPUT

```
1000
1000.0
4.0
```

An `int` combined with a `float` gives a `float`, which is why the subtotal earlier was `1000.0`. Division with `/` always gives a `float`, even when the answer is a whole number.

Tip: Use brackets to make your intention obvious: `subtotal * (discount_pct / 100)` reads more clearly than `subtotal * discount_pct / 100`, even though both give `100.0`. If a line feels hard to read, split it into named steps.

Comparisons produce True or False

Operator	Question it asks
==	Are the two values equal?
!=	Are they different?
> <	Greater than? Less than?
>= <=	Greater than or equal? Less than or equal?

p4_compare.py

```
quantity = 4
final_total = 945.0
large_order_threshold = 950.0

print(quantity == 4)
print(quantity != 4)
print(final_total >= large_order_threshold)
```

OUTPUT

```
True
False
False
```

= assigns, == asks

Code	Say it as	Result
<code>quantity = 4</code>	"Assign 4 to quantity." An instruction.	nothing printed; the name now refers to 4
<code>quantity == 4</code>	"Is quantity equal to 4?" A question.	<code>True</code> or <code>False</code>

`True` and `False` (capital first letter, no quotes) are `bool` values. `"False"` in quotes is just a five-letter piece of text.

and, or, not

`p4_logic.py`

```
unit_price = 250.0
quantity = 4

price_is_positive = unit_price > 0
quantity_is_positive = quantity > 0
inputs_look_positive = price_is_positive and quantity_is_positive

print(inputs_look_positive)
print(not inputs_look_positive)
print(quantity > 10 or unit_price > 100)
```

OUTPUT

```
True
False
True
```

`and` is `True` only when both sides are `True`. `or` is `True` when at least one side is. `not` flips `True` and `False`. We are only *producing* booleans today; making the program choose what to do with them is the `if` statement, in Session 08.

TRY IT YOURSELF

Create `p4_coffee.py` for the café order from Part 1: price **32.50**, **3** cups, **10%** tip.

(a) Calculate `subtotal`, `tip_amount` and `total` and print each one. (b) Create `is_big_order`, which is `True` when the total is at least 100, and print it. (c) Print the type of `subtotal` and of `is_big_order`.

Part 5 Input and conversion

So far every value was typed into the code. Real programs get values from outside, for example from the keyboard with `input()`.

`input()` always gives you text

Create `p5_input.py`, run it, and type `4` when asked. The program waits in the terminal until you press Enter.

p5_input.py

```
quantity_text = input("Quantity: ")
print(quantity_text)
print(type(quantity_text))
```

TERMINAL

```
> python p5_input.py
Quantity: 4
4
<class 'str'>
```

You typed a number, but the program received the **text** "4". `input()` always returns a `str`. The program has to decide what the text means.

Forgetting to convert

p5_no_convert.py

```
quantity_text = input("Quantity: ")
subtotal = quantity_text * 250.0
print(subtotal)
```

TERMINAL

```
> python p5_no_convert.py
Quantity: 4
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\p5_no_convert.py", line 2, in <module>
    subtotal = quantity_text * 250.0
                ~~~~~^~~~~~
TypeError: can't multiply sequence by non-int of type 'float'
```

The same `TypeError` as in Part 4: text times a decimal. The fix is to convert first.

int(), float() and str()

Function	Does	Example
<code>int(...)</code>	makes a whole number from suitable text	<code>int("4")</code> → 4
<code>float(...)</code>	makes a decimal from suitable text	<code>float("250.0")</code> → 250.0
<code>str(...)</code>	makes text from a value	<code>str(4)</code> → "4"

p5_convert.py

```

quantity_text = input("Quantity: ")
unit_price_text = input("Unit price: ")

quantity = int(quantity_text)
unit_price = float(unit_price_text)

print(quantity, type(quantity))
print(unit_price, type(unit_price))
print("Subtotal:", unit_price * quantity)

```

TERMINAL

```

> python p5_convert.py
Quantity: 4
Unit price: 250.0
4 <class 'int'>
250.0 <class 'float'>
Subtotal: 1000.0

```

Once you trust the pattern, input and conversion are usually written on one line: `quantity = int(input("Quantity: "))`. Read it **from the inside out**:

Order	Part	What happens
1	<code>input("Quantity: ")</code>	shows the prompt and returns the typed text, e.g. "4"
2	<code>int(...)</code>	turns that text into the whole number 4
3	<code>quantity = ...</code>	stores the number under the name <code>quantity</code>

str() goes the other way

p5_str.py

```

quantity = 4
print("MA" + str(quantity))
print("MA" + quantity)

```

OUTPUT

MA4

ERROR

```

Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\p5_str.py", line 3, in <module>
    print("MA" + quantity)
            ~~~~~^~~~~~
TypeError: can only concatenate str (not "int") to str

```

`+` joins text to text. To attach a number to text, convert it with `str()` first. We will need exactly this for the order code.

When conversion fails

Run `p5_convert.py` again and type `four` for the quantity:

p5_value_error.py

```
quantity_text = input("Quantity: ")
quantity = int(quantity_text)
print(quantity)
```

TERMINAL

```
> python p5_value_error.py
Quantity: four
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\p5_value_error.py", line 2, in <module>
    quantity = int(quantity_text)
ValueError: invalid literal for int() with base 10: 'four'
```

A **ValueError** means the operation was correct (converting text to a number) but this particular value cannot be converted. It also catches a common surprise: typing `4.5` for an `int`.

p5_int_decimal.py

```
print(int("4.5"))
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\p5_int_decimal.py", line 1, in <module>
    print(int("4.5"))
    ~~~~~^
ValueError: invalid literal for int() with base 10: '4.5'
```

Error	Usually means	Example
<code>TypeError</code>	the operation got the wrong <i>kind</i> of value	<code>"4" * 250.0</code>
<code>ValueError</code>	the kind is right, but this <i>value</i> cannot be used	<code>int("four"), float("abc")</code>

Trainer note: Learners will ask how to stop bad input crashing the program. Acknowledge it and park it: checking input and handling errors with `try/except` is Session 08. Today the skill is reading the error and correcting the input.

TRY IT YOURSELF

Create `p5_coffee_input.py`. Ask for the coffee price and the number of cups, convert both, and print the subtotal.

Test it with `32.50` and `3`. You should see `Subtotal: 97.5`. Then run it again and type `three` for the cups. Which error do you get, and on which line?

Part 6 Strings and indexing

Names typed by people are messy: extra spaces, lowercase, inconsistent capitals. We need a few string tools to tidy them up and to build an order code.

Cleaning text

p6_clean.py

```
customer_name = "  maya patel  "
print "[" + customer_name + "]"

clean_name = customer_name.strip()
print "[" + clean_name + "]"
print(clean_name.title())
print(clean_name.upper())
```

OUTPUT

```
[  maya patel  ]
[maya patel]
Maya Patel
MAYA PATEL
```

The square brackets are only there so you can see the spaces. `.strip()` removes spaces at the start and end (not the space between words). `.title()` capitalises each word and `.upper()` capitalises everything.

These are **methods**: tools that belong to a value and are called with a dot. Methods can be chained, and they run from left to right:

p6_chain.py

```
customer_name = "  maya patel  "
customer_name = customer_name.strip().title()
print(customer_name)
print(len(customer_name))
```

OUTPUT

```
Maya Patel
10
```

`len()` counts characters. `Maya Patel` has 10, because the space counts.

Watch out: `customer_name.strip()` on its own does not change `customer_name`. It produces a new, cleaned value. To keep it, assign it: `customer_name = customer_name.strip()`.

Indexing: picking one character

Each character has a position, and **Python starts counting at 0**. Negative positions count back from the end.

M	a	y	a		P	a	t	e	l	character
0	1	2	3	4	5	6	7	8	9	position from the start
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	position from the end

PREDICT FIRST

What does each line print? Pay attention to index 0 and index -1.

p6_index.py

```
customer_name = "Maya Patel"
product_name = "Analytics Starter Kit"

print(customer_name[0])
print(customer_name[1])
print(customer_name[-1])
print(customer_name[-2])
print(product_name[0])
print(product_name[-1])
```

OUTPUT

```
M
a
l
e
A
t
```

Watch out: The first character is `[0]`, not `[1]`. Asking for a position that does not exist, such as `customer_name[10]`, stops the program with `IndexError: string index out of range`.

Readable output with f-strings

p6_fstring.py

```
final_total = 945.0

print("Final total:", final_total)
print(f"Final total: {final_total}")
print(f"Final total: {final_total:.2f}")
```

OUTPUT

```
Final total: 945.0
Final total: 945.0
Final total: 945.00
```

Put an `f` before the opening quote and Python fills in anything inside `{ }`. Adding `:.2f` shows the number with two decimal places. This only changes how the number is **displayed**; the stored value is still `945.0`.

Building the order code

p6_order_code.py

```
customer_name = "Maya Patel"
product_name = "Analytics Starter Kit"
quantity = 4

order_code = customer_name[0].upper() + product_name[0].upper() + str(quantity)
print(f"Order code: {order_code}")
```

OUTPUT

```
Order code: MA4
```

One line, six ideas from today: looking up a variable, zero-based indexing, a string method, converting an int with `str()`, joining text with `+`, and assignment.

Trainer note: Ask learners to name each of the six ideas in that line before you do. It is a quick check of everything since the break.

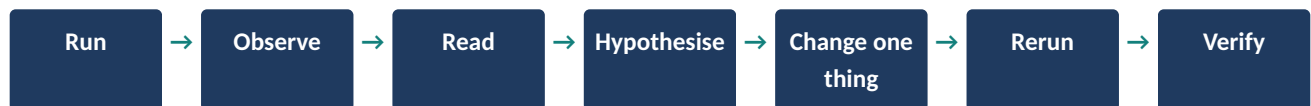
TRY IT YOURSELF

Create `p6_customer.py` starting with `customer = " thandi nkosi "`, `product = "flat white"` and `cups = 3`.

(a) Clean both names with `.strip().title()` and print them. (b) Print the length of the cleaned customer name. (c) Print its first and last characters. (d) Build and print an order code made of the first letter of the customer, the first letter of the product and the number of cups. Expected: `TF3`.

Part 7 Reading errors and debugging

Everyone who writes code sees errors every day. An error message is not a verdict on you; it is **evidence** about what Python could not do. Debugging means using that evidence, in this order every time:



Changing several lines at once makes it impossible to know which change fixed the problem, or which one broke something else.

Reading a traceback from the bottom up

`p7_traceback.py`

```
final_total = 945.0
print(final_totl)
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\p7_traceback.py", line 2, in <module>
    print(final_totl)
    ^^^^^^^^^^^
NameError: name 'final_totl' is not defined. Did you mean: 'final_total'?
```

Read	Here	Tells you
1. The last line	<code>NameError: name 'final_totl' is not defined</code>	what kind of error, and the message
2. The File line	<code>line 2</code>	where it happened; click the path in VS Code to jump there
3. The code shown	<code>print(final_totl)</code> and the <code>^</code> marks	exactly which part Python objected to

Five debugging stations

Each station is a starter file in the class bundle. Run it, read the error, say the error type and its likely cause out loud, then make **one** fix and run again.

Station 1: SyntaxError

debug_station_1.py

```
print("Quote ready)
```

ERROR

```
File "C:\Users\student\s07_python_basics\debug_station_1.py", line 1
    print("Quote ready)
        ^
SyntaxError: unterminated string literal (detected at line 1)
```

Python could not even read the line: the text has an opening quote but no closing quote. Nothing in the file runs when there is a syntax error. **Fix:** `print("Quote ready")`

Station 2: IndentationError

debug_station_2.py

```
subtotal = 1000.0
    final_total = 945.0
print(final_total)
```

ERROR

```
File "C:\Users\student\s07_python_basics\debug_station_2.py", line 2
    final_total = 945.0
IndentationError: unexpected indent
```

Leading spaces mean something in Python, so an unexpected indent is an error. **Fix:** move `final_total = 945.0` back to the left margin.

Station 3: NameError

debug_station_3.py

```
final_total = 945.0
print(final_totl)
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\debug_station_3.py", line 2, in <module>
    print(final_totl)
          ^^^^^^^^^
NameError: name 'final_totl' is not defined. Did you mean: 'final_total'?
```

Python has no value under that name. Either the name is misspelled, or the line that creates it has not run yet (common with Shift+Enter or notebooks). **Fix:** `print(final_total)`

Station 4: TypeError

debug_station_4.py

```
quantity = "4"
subtotal = quantity * 250.0
print(subtotal)
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\debug_station_4.py", line 2, in <module>
    subtotal = quantity * 250.0
               ~~~~~~^~~~~~
TypeError: can't multiply sequence by non-int of type 'float'
```

The operation received a kind of value it cannot use: text where a number was needed. **Fix:** `quantity = int("4")`, or convert with `int(quantity)` before multiplying.

Station 5: ValueError

debug_station_5.py

```
unit_price = float("abc")
print(unit_price)
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\debug_station_5.py", line 1, in <module>
    unit_price = float("abc")
ValueError: could not convert string to float: 'abc'
```

Converting to a float is fine, but `"abc"` is not a number. **Fix:** supply valid numeric text, e.g. `float("250.0")`.

Not every bug shows an error

A program can run without any error and still be wrong: a `+` where you meant `-`, or a REPL holding an old value (Part 2). That is why we check intermediate values against a hand calculation, and finish with a clean top-to-bottom run.

Trainer note: Give learners a minute to read each error themselves. Step in when someone is changing several things at once, rerunning without reading, or stuck on a setup issue rather than the code.

TRY IT YOURSELF

Open `try_it_debug.py` from the bundle. It has **three** bugs. Fix them **one at a time**, running after each fix. For each, write down the error type and the line number before you change anything.

When it is working it prints `Total: 97.50`.

try_it_debug.py

```
coffee_price = float("32.50")
cups = "3"
total = coffee_price * cups
print(f"Total: {totl:.2f}")
```

Part 8 Build the quote calculator

Now everything comes together in one file. From here the learners do the work; the trainer coaches.

Trainer note: Do not show the finished code before learners have tried. When someone's total is wrong, ask for their `subtotal`, then `discount_amount`, and so on until you find the **first** value that is wrong. That is where the bug is.

The inputs

Variable	Value for testing	Type after conversion
<code>customer_name</code>	" maya patel "	str (cleaned to Maya Patel)
<code>product_name</code>	"analytics starter kit"	str (cleaned to Analytics Starter Kit)
<code>unit_price</code>	"250.0"	float
<code>quantity</code>	"4"	int
<code>discount_pct</code>	"10.0"	float
<code>tax_pct</code>	"5.0"	float
<code>large_order_threshold</code>	950.0	float (fixed by the business, not typed in)

The numbers are given as text on purpose: that is exactly what `input()` will hand you later. Starting with fixed values means you can test the maths first and add the keyboard last.

The calculation, in order

Step	Expression	Checkpoint
<code>subtotal</code>	<code>unit_price * quantity</code>	1000.0
<code>discount_amount</code>	<code>subtotal * (discount_pct / 100)</code>	100.0
<code>discounted_subtotal</code>	<code>subtotal - discount_amount</code>	900.0
<code>tax_amount</code>	<code>discounted_subtotal * (tax_pct / 100)</code>	45.0
<code>final_total</code>	<code>discounted_subtotal + tax_amount</code>	945.0
<code>is_large_order</code>	<code>final_total >= large_order_threshold</code>	False
<code>order_code</code>	first letter of customer + first letter of product + quantity	MA4

Each line uses the result of the line before. If one value is wrong, everything after it will be wrong too.

Step 1: open the starter file

Open `quote_calculator_starter.py` from the bundle, save a copy as `quote_calculator.py`, and replace every `...` with the right expression. Run after **each** section, not just at the end.

quote_calculator_starter.py

```

# Sales Quote & Order Summary Utility -- starter file
# Replace each ... with an expression. Run after every section.

# ---- 1. Inputs (as text, exactly as input() would give them) ----
raw_customer_name = " maya patel "
raw_product_name = "analytics starter kit"
raw_unit_price = "250.0"
raw_quantity = "4"
raw_discount_pct = "10.0"
raw_tax_pct = "5.0"
large_order_threshold = 950.0

# ---- 2. Clean and convert ----
customer_name = ...      # strip and title raw_customer_name
product_name = ...      # strip and title raw_product_name
unit_price = ...         # convert to float
quantity = ...           # convert to int
discount_pct = ...       # convert to float
tax_pct = ...            # convert to float

# ---- 3. Calculate, in this order ----
subtotal = ...
discount_amount = ...
discounted_subtotal = ...
tax_amount = ...
final_total = ...
is_large_order = ...
order_code = ...

# ---- 4. Summary ----
print("SALES QUOTE & ORDER SUMMARY")
print("-----")
print(f"Order code: {order_code}")
print(f"Customer: {customer_name}")
print(f"Product: {product_name}")
print(f"Unit price: {unit_price:.2f}")
print(f"Quantity: {quantity}")
print(f"Subtotal: {subtotal:.2f}")
print(f"Discount ({discount_pct:.1f}%): {discount_amount:.2f}")
print(f"Discounted subtotal: {discounted_subtotal:.2f}")
print(f"Tax ({tax_pct:.1f}%): {tax_amount:.2f}")
print(f"Final total: {final_total:.2f}")
print(f"Large order threshold: {large_order_threshold:.2f}")
print(f"Is large order: {is_large_order}")

```

Tip: While building, add temporary checks such as `print("subtotal:", subtotal)` straight after a calculation, and compare with the checkpoint table. Delete them when the summary is right.

Step 2: check against the expected summary

When every `...` is replaced, running the file should print exactly this:

OUTPUT

```
SALES QUOTE & ORDER SUMMARY
-----
Order code: MA4
Customer: Maya Patel
Product: Analytics Starter Kit
Unit price: 250.00
Quantity: 4
Subtotal: 1000.00
Discount (10.0%): 100.00
Discounted subtotal: 900.00
Tax (5.0%): 45.00
Final total: 945.00
Large order threshold: 950.00
Is large order: False
```

Step 3: defect hunt

These four mistakes are the ones learners make most often in this file. For each, **predict the error type before running it**, then put the mistake into your working file, run, and compare. Undo it before moving to the next one. Answers are in the appendix.

#	Change this line to	Error type?
A	<code>quantity = raw_quantity</code> (conversion forgotten)	
B	<code>final_total = discounted_subtotal + tax_amont</code>	
C	<code>raw_unit_price = "250,0"</code> (comma instead of point)	
D	<code>tax_amount = discounted_subtotal * (tax_pct / 100)</code> (four spaces in front)	

Step 4: switch to real keyboard input

Once the summary matches, replace the six `raw_` lines with `input()` calls. Everything below them stays exactly the same, because the rest of the program only ever sees text, whether you typed it into the code or at the keyboard.

```
raw_customer_name = input("Customer name: ")
raw_product_name = input("Product name: ")
raw_unit_price = input("Unit price: ")
raw_quantity = input("Quantity: ")
raw_discount_pct = input("Discount percentage: ")
raw_tax_pct = input("Tax percentage: ")
```

TERMINAL

```
> python quote_calculator.py
Customer name:  maya patel
Product name: analytics starter kit
Unit price: 250
Quantity: 4
Discount percentage: 10
Tax percentage: 5
SALES QUOTE & ORDER SUMMARY
-----
Order code: MA4
Customer: Maya Patel
Product: Analytics Starter Kit
Unit price: 250.00
Quantity: 4
Subtotal: 1000.00
Discount (10.0%): 100.00
Discounted subtotal: 900.00
Tax (5.0%): 45.00
Final total: 945.00
Large order threshold: 950.00
Is large order: False
```

Notice that typing `250` (no decimal point) still works, because `float("250")` gives `250.0`. Try a second customer of your own and check the total with a calculator.

Step 5: the final clean run

Close any REPL terminals (bin icon), save the file, and run it with ► from a fresh terminal. If it works now, it works because of what is in the file, not because of something left over from earlier. This is the same idea as *Restart and Run All* in a notebook.

Part 9 Variation and close

Add a service fee

The business now adds a fixed service fee of **25.00** after tax. Add these three lines to the calculation section and two lines to the summary:

```
service_fee = 25.0
final_total_with_service = final_total + service_fee
is_large_order_with_service = final_total_with_service >= large_order_threshold
```

PREDICT FIRST

With the test values, what will `final_total_with_service` and `is_large_order_with_service` be?

The answer (in the appendix) shows something important: the large-order result can change from `False` to `True` purely because the data changed. No `if` statement was needed. Making the program *do something different* when an order is large is where Session 08 begins.

TRY IT YOURSELF

- (a) Print the two new values with f-strings, the total to two decimal places.
- (b) Without running anything, work out the final total, the large-order result and the order code if the quantity is **3** instead of 4 (no service fee). Then run the program with 3 and check.

Explain your program

Before leaving, each learner points to one example of each of these in **their own** `quote_calculator.py`:

- ☐ an **input**
- ☐ an **expression**
- ☐ an **assignment** statement
- ☐ a **type conversion**
- ☐ a **string operation**
- ☐ a **boolean comparison**
- ☐ a **debugging decision** you made from an error message
- ☐ evidence that the file runs **cleanly from top to bottom**

Questions to park for later sessions

If a learner asks...	It's covered in
Can it do something different when the order is large?	Session 08: <code>if / elif / else</code>
Can we repeat this for many orders?	Session 08: loops
Can we package the calculation to reuse it?	Session 08: functions
How do we stop bad input from crashing it?	Session 08: <code>try / except</code>
Can we keep many products or orders together?	Session 09: lists and dictionaries
Can we save quotes to a file?	Session 09: files and JSON
When do we use Pandas?	Sessions 11–12

Close with: A program turns a problem into explicit steps. Values have types, expressions produce values, names keep them, operators transform or compare them, input arrives as text and may need converting, and errors tell us where to look next.

Appendix: Answers to the exercises

Part 1

Inputs: coffee price (32.50), number of cups (3), tip percentage (10). **Steps:** subtotal = price × cups; tip = subtotal × 10/100; total = subtotal + tip. **Outputs:** subtotal, tip, total. **By hand:** $32.50 \times 3 = 97.50$; tip = 9.75; total = **107.25**.

Part 3

answer_p3.py

```
cups = 2
cups = cups + 3
print(cups)
print("cups")
```

OUTPUT

```
5
cups
```

(b) Valid: `coffee_price`, `cups3`, `Cups` (valid, but a different name from `cups` and not snake_case). Not valid: `coffee price` (space), `3cups` (starts with a digit), `class` (keyword).

Part 4

p4_coffee.py

```
coffee_price = 32.50
cups = 3
tip_pct = 10.0

subtotal = coffee_price * cups
tip_amount = subtotal * (tip_pct / 100)
total = subtotal + tip_amount
is_big_order = total >= 100

print(subtotal)
print(tip_amount)
print(total)
print(is_big_order)
print(type(subtotal))
print(type(is_big_order))
```

OUTPUT

```
97.5
9.75
107.25
True
<class 'float'>
<class 'bool'>
```

Part 5

p5_coffee_input.py

```
coffee_price = float(input("Coffee price: "))
cups = int(input("Number of cups: "))
print("Subtotal:", coffee_price * cups)
```

TERMINAL

```
> python p5_coffee_input.py
Coffee price: 32.50
Number of cups: 3
Subtotal: 97.5
```

Typing `three` for the cups gives the error below, on line 2 (the `int(...)` conversion):

p5_coffee_input.py

```
coffee_price = float(input("Coffee price: "))
cups = int(input("Number of cups: "))
print("Subtotal:", coffee_price * cups)
```

TERMINAL

```
> python p5_coffee_input.py
Coffee price: 32.50
Number of cups: three
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s07_python_basics\p5_coffee_input.py", line 2, in <module>
    cups = int(input("Number of cups: "))
ValueError: invalid literal for int() with base 10: 'three'
```

Part 6

p6_customer.py

```
customer = "  thandi nkosi  "
product = "flat white"
cups = 3

customer = customer.strip().title()
product = product.strip().title()
print(customer)
print(product)
print(len(customer))
print(customer[0])
print(customer[-1])

order_code = customer[0].upper() + product[0].upper() + str(cups)
print(order_code)
```

OUTPUT

```
Thandi Nkosi
Flat White
12
T
i
TF3
```

Part 7: try_it_debug.py, one bug at a time

Error shown (last line)	Line	Fix
Run 1: <code>IndentationError: unexpected indent</code>	3	remove the spaces before <code>total</code>
Run 2: <code>TypeError: can't multiply sequence by non-int of type 'float'</code>	3	<code>cups = int("3")</code>
Run 3: <code>NameError: name 'totl' is not defined. Did you mean: 'total'?</code>	4	<code>{total:.2f}</code>

Notice that Python reports only the first problem it meets. The indentation error is found before anything runs; the `TypeError` and `NameError` only appear once the lines above them work.

`try_it_debug_fixed.py`

```
coffee_price = float("32.50")
cups = int("3")
total = coffee_price * cups
print(f"Total: {total:.2f}")
```

OUTPUT

```
Total: 97.50
```

Part 8: defect hunt

#	Error shown (last line)	Why
A	<code>TypeError: can't multiply sequence by non-int of type 'float'</code>	<code>quantity</code> is still text, so <code>unit_price * quantity</code> is <code>float × str</code> .
B	<code>NameError: name 'tax_amont' is not defined. Did you mean: 'tax_amount'?</code>	Misspelled name: <code>tax_amont</code> was never created.
C	<code>ValueError: could not convert string to float: '250,0'</code>	<code>float()</code> only understands a point as the decimal separator.
D	<code>IndentationError: unexpected indent</code>	An unexpected indent; the file does not run at all.

Watch out: Defect C is worth a minute in a South African class: many local settings write decimals with a comma (250,0). Python always expects a point.

Part 9

`final_total_with_service` = 945.0 + 25.0 = **970.0**, and because $970.0 \geq 950.0$, `is_large_order_with_service` is **True**.

With quantity 3: subtotal 750.0 → discount 75.0 → discounted subtotal 675.0 → tax 33.75 → final total **708.75**; large order **False**; order code **MA3**.

Completed quote_calculator.py (keyboard version)

This is the finished program from Part 8, Step 4, without the service fee. It is also in the class bundle as `quote_calculator_input.py`.

quote_calculator_input.py

```

# Sales Quote & Order Summary Utility -- completed (keyboard input)

# ---- 1. Inputs from the keyboard (always text) ----
raw_customer_name = input("Customer name: ")
raw_product_name = input("Product name: ")
raw_unit_price = input("Unit price: ")
raw_quantity = input("Quantity: ")
raw_discount_pct = input("Discount percentage: ")
raw_tax_pct = input("Tax percentage: ")
large_order_threshold = 950.0

# ---- 2. Clean and convert ----
customer_name = raw_customer_name.strip().title()
product_name = raw_product_name.strip().title()
unit_price = float(raw_unit_price)
quantity = int(raw_quantity)
discount_pct = float(raw_discount_pct)
tax_pct = float(raw_tax_pct)

# ---- 3. Calculate, in this order ----
subtotal = unit_price * quantity
discount_amount = subtotal * (discount_pct / 100)
discounted_subtotal = subtotal - discount_amount
tax_amount = discounted_subtotal * (tax_pct / 100)
final_total = discounted_subtotal + tax_amount
is_large_order = final_total >= large_order_threshold
order_code = customer_name[0].upper() + product_name[0].upper() + str(quantity)

# ---- 4. Summary ----
print("SALES QUOTE & ORDER SUMMARY")
print("-----")
print(f"Order code: {order_code}")
print(f"Customer: {customer_name}")
print(f"Product: {product_name}")
print(f"Unit price: {unit_price:.2f}")
print(f"Quantity: {quantity}")
print(f"Subtotal: {subtotal:.2f}")
print(f"Discount ({discount_pct:.1f}%): {discount_amount:.2f}")
print(f"Discounted subtotal: {discounted_subtotal:.2f}")
print(f"Tax ({tax_pct:.1f}%): {tax_amount:.2f}")
print(f"Final total: {final_total:.2f}")
print(f"Large order threshold: {large_order_threshold:.2f}")
print(f"Is large order: {is_large_order}")

```